

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验 11 设备管理与 SPOOLING]

姓名：	顾凯琪
学号：	226001624
课程名称：	操作系统
所在学院：	信息科学与工程学院
专业班级：	22 计算机科学与技术 2 班
任课教师：	江先亮

实验项目名称	实验 11 设备管理与 SPOOLING

一、实验目的与要求：

(1)了解 openEuler 设备管理和 SPOOLING 原理

(2)基于 openEuler 系统分析设备管理和 SPOOLING

二、实验环境：

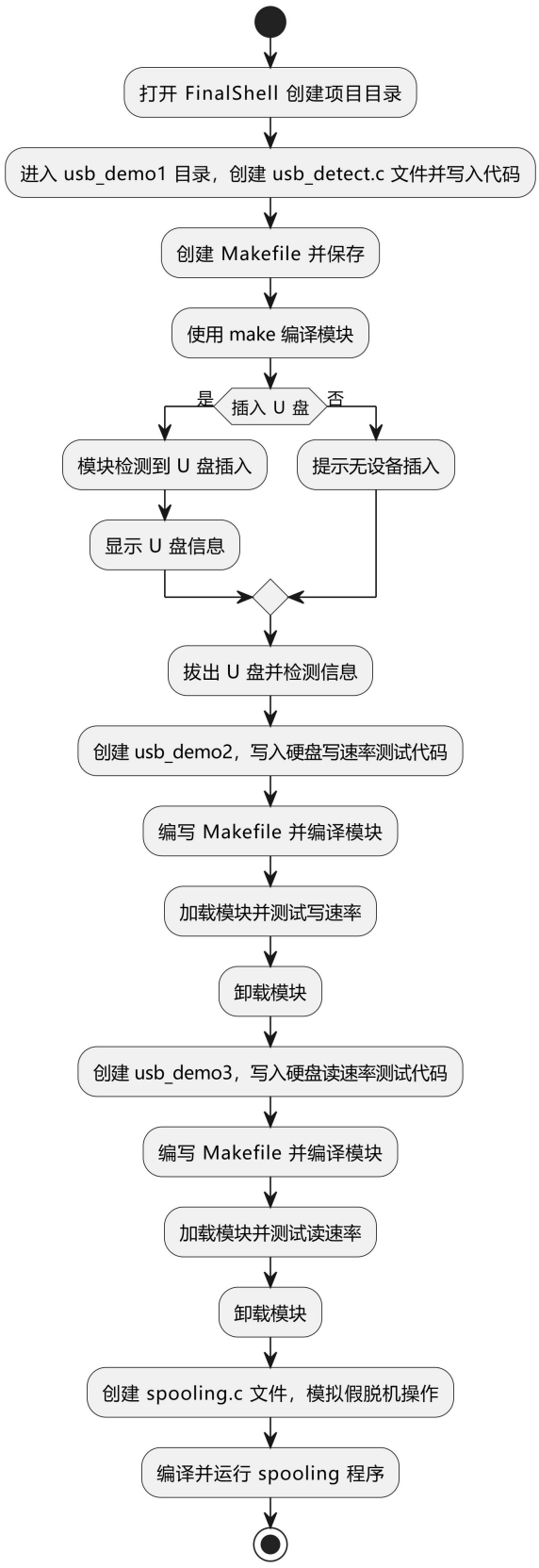
Win11 OS

VMware workstation 17 pro

OpenEular 22.03

FinalShell 4.5.6

三、实验方法（原理、流程图）


打开 FinalShell 创建项目目录
进入 usb_demo1 目录，创建 usb_detect.c 文件并写入代码
创建 Makefile 并保存
使用 make 编译模块
插入 U 盘
是
否
模块检测到 U 盘插入
提示无设备插入
显示 U 盘信息
拔出 U 盘并检测信息
创建 usb_demo2，写入硬盘写速率测试代码
编写 Makefile 并编译模块
加载模块并测试写速率
卸载模块
创建 usb_demo3，写入硬盘读速率测试代码
编写 Makefile 并编译模块
加载模块并测试读速率
卸载模块
创建 spooling.c 文件，模拟假脱机操作
编译并运行 spooling 程序

四、实验过程、步骤及内容

USB 驱动的注册和注销

参考内核源码中的 **drivers/usb/usb-skeleton.c** 文件，编写一个 **USB** 探测驱动 程序，能够实现基本功能：（**1**）在插入 **U** 盘时能够探测到；（**2**）在拔出 **U** 盘时能够探测到。加载、卸载模块并查看模块打印信息。

① 打开 **FinalShell** 窗口，创建一个目录，命名为 **usb_demo1,cd** 到该目录下，创建一个 **C** 文件，命名为 **usb_detect**，并打开该文件：

```
[root@localhost ~]# mkdir usb_demo1
[root@localhost ~]# ls
anaconda-ks.cfg  b'  exp  exp1  exp2  exp3  exp4  e
[root@localhost ~]# cd usb_demo1/
[root@localhost usb_demo1]# touch usb_detect.c
[root@localhost usb_demo1]# vi usb_detect.c
```

②按“**i**”键进入 **Insert** 模式，在 **usb_detect.c** 文件中写入如下代码，按“**Esc**”退出 **Insert** 模式，输入“**:wq**”保存：

```
#include <linux/kernel.h>
#include <linux/errno.h>
#include <linux/slab.h>
#include <linux/module.h>
#include <linux/kref.h>
#include <linux/uaccess.h>
#include <linux/usb.h>
#include <linux/mutex.h>

/* Define these values to match your devices */
#define USB_DETECT_VENDOR_ID 0x090c
#define USB_DETECT_PRODUCT_ID 0x2000

/* Table of devices that work with this driver */
static const struct usb_device_id usbdetect_table[] = {
    { USB_DEVICE(USB_DETECT_VENDOR_ID, USB_DETECT_PRODUCT_ID) },
    {} /* Terminating entry */
};
MODULE_DEVICE_TABLE(usb, usbdetect_table);

/* Get a minor range for your devices from the USB maintainer */
#define USB_DETECT_MINOR_BASE 192
#define WRITES_IN_FLIGHT 8 /* Arbitrarily chosen */

/* Structure to hold all of our device-specific data */
struct usb_detect {
    struct usb_device *udev;              /* The USB device */
    struct usb_interface *interface;      /* The interface */
    struct semaphore limit_sem;           /* Limit writes in progress */
    struct usb_anchor submitted;          /* To retract submissions */
    struct urb *bulk_in_urb;              /* URB for reading data */
    unsigned char *bulk_in_buffer;        /* Buffer for receiving data */
    size_t bulk_in_size;                  /* Size of the receive buffer */
    size_t bulk_in_filled;                /* Bytes in the buffer */
    size_t bulk_in_copied;                /* Bytes copied to user space */
    __u8 bulk_in_endpointAddr;            /* Bulk-in endpoint address */
    __u8 bulk_out_endpointAddr;           /* Bulk-out endpoint address */
    int errors;                           /* Last request error */
    bool ongoing_read;                    /* Read is ongoing */
    spinlock_t err_lock;                  /* Lock for errors */
    struct kref kref;                     /* Reference counter */
:wq
```

③编写 **Makefile** 文件，按“**i**”键进入 **Insert** 模式，在文件中写入如下代码，按“**Esc**”退出 **Insert** 模式，输入“：**wq**”保存：

```
ifneq ($(KERNELRELEASE),)
    obj-m := usb_detect.o
else
    # 选择自己的内核路径
    KERNELDIR ?= /root/kernel-openEuler-22.03-LTS-SP2
    PWD := $(shell pwd)
default:
        $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif
.PHONY: clean
clean:
        -rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 **U** 盘内核模块 ：

```
[root@localhost usb_demo1]# make
make -C /root/kernel-openEuler-22.03-LTS-SP2 M=/root/usb_demo1 modules
make[1]: 进入目录“/root/kernel-openEuler-22.03-LTS-SP2”
  CC [M]  /root/usb_demo1/usb_detect.o
  MODPOST /root/usb_demo1/Module.symvers
  CC [M]  /root/usb_demo1/usb_detect.mod.o
  LD [M]  /root/usb_demo1/usb_detect.ko
make[1]: 离开目录“/root/kernel-openEuler-22.03-LTS-SP2”
```

⑤在电脑的 USB 接口插入 U 盘后，查看信息可见，模块检测到 U 盘插入（如最后一行所示），并且获取到的 U 盘信息与 lsusb 显示的一致

```
[root@localhost usb_demo1]# insmod usb_detect.ko
[root@localhost usb_demo1]# dmesg -C
[root@localhost usb_demo1]# dmesg  | tail -n 20
[root@localhost usb_demo1]# dmesg  | tail -n 20
[13941.623349] usb 1-1: new high-speed USB device number 2 using ehci-pci
[13941.815811] usb 1-1: Invalid ep0 maxpacket: 9
[13942.044823] usb 1-1: new high-speed USB device number 3 using ehci-pci
[13942.283682] usb 1-1: Invalid ep0 maxpacket: 9
[13942.284213] usb usb1-port1: attempt power cycle
[13942.572730] usb 1-1: new high-speed USB device number 4 using ehci-pci
[13947.620501] usb 1-1: device descriptor read/8, error -110
[root@localhost usb_demo1]#
```

⑥从电脑中拔出 U 盘,查看打印信息可见:USB 设备已断开连接(最后一行所示):

```
[13941.623349] usb 1-1: new high-speed USB device number 2 using ehci-pci
[13941.815811] usb 1-1: Invalid ep0 maxpacket: 9
[13942.044823] usb 1-1: new high-speed USB device number 3 using ehci-pci
[13942.283682] usb 1-1: Invalid ep0 maxpacket: 9
[13942.284213] usb usb1-port1: attempt power cycle
[13942.572730] usb 1-1: new high-speed USB device number 4 using ehci-pci
[13947.620501] usb 1-1: device descriptor read/8, error -110
[13963.492186] usb 1-1: device descriptor read/8, error -110
[13963.718842] usb 1-1: new high-speed USB device number 5 using ehci-pci
[13969.125506] usb 1-1: device descriptor read/8, error -110
[13984.996465] usb 1-1: device descriptor read/8, error -110
[13985.105113] usb usb1-port1: unable to enumerate USB device
[root@localhost usb_demo1]#
```

⑦可以用 lsusb 命令查看 U 盘插入和拔出的情况(输出信息的第一行):

```
Complete!
[root@localhost usb_demo1]# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 004: ID 0e0f:0008 VMware, Inc. Virtual Bluetooth Adapter
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
[root@localhost usb_demo1]#
```

⑧卸载模块：

```
[root@localhost usb_demo1]# rmmod usb_detect.ko
[root@localhost usb_demo1]# dmesg |tail -n 20
[13941.623349] usb 1-1: new high-speed USB device number 2 using ehci-pci
[13941.815811] usb 1-1: Invalid ep0 maxpacket: 9
[13942.044823] usb 1-1: new high-speed USB device number 3 using ehci-pci
[13942.283682] usb 1-1: Invalid ep0 maxpacket: 9
[13942.284213] usb usb1-port1: attempt power cycle
[13942.572730] usb 1-1: new high-speed USB device number 4 using ehci-pci
[13947.620501] usb 1-1: device descriptor read/8, error -110
[13963.492186] usb 1-1: device descriptor read/8, error -110
[13963.718842] usb 1-1: new high-speed USB device number 5 using ehci-pci
[13969.125506] usb 1-1: device descriptor read/8, error -110
[13984.996465] usb 1-1: device descriptor read/8, error -110
[13985.105113] usb usb1-port1: unable to enumerate USB device
[14121.946233] capability: warning: `yum' uses 32-bit capabilities (legacy support in use)
[14269.788102] Exit usb_detect module...
[14269.788105] usbcore: deregistering interface driver usbdetect
[root@localhost usb_demo1]#
```

内核文件编写模拟

①先编写内核模块测试硬盘的写速率。打开 **FinalShell** 窗口，创建一个目录，命名为 **usb_demo2,cd** 到该目录下，创建一个 **C** 文件，命名为 **write_to_disk**，并打开该文件：

```
[root@localhost ~]# mkdir usb_demo2
[root@localhost ~]# cd usb_demo2
[root@localhost usb_demo2]# touch write_to_disk
[root@localhost usb_demo2]# vi write_to_disk
```

②按“**i**”键进入 **Insert** 模式，在 **write_to_disk.c** 文件中写入如下代码，按“**Esc**”退出 **Insert** 模式，输入“：**wq**”保存：

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/rtc.h>
#define buf_size 1024
#define write_times 524288
MODULE_LICENSE("GPL");
struct timeval tv;
static int __init write_disk_init(void)
{
struct file *fp_write;
char buf[buf_size];
int i;
int write_start_time;
int write_start_time_u;
int write_end_time;
int write_end_time_u;
int write_time;
loff_t pos;
printk("Start write_to_disk module...\n");
for(i = 0; i < buf_size; i++)
{
buf[i] = i + '0';
}
fp_write = filp_open("/home/tmp_file", O_RDWR | O_CREAT,0644);
if (IS_ERR(fp_write)) {
printk("Failed to open file...\n");
return -1;
}
pos = 0;
do_gettimeofday(&tv);
write_start_time = (int)tv.tv_sec;
write_start_time_u = (int)tv.tv_usec;
for(i = 0; i < write_times; i++) {
kernel_write(fp_write, buf, buf_size, &pos);
}
do_gettimeofday(&tv);
write_end_time = (int)tv.tv_sec;
write_end_time_u = (int)tv.tv_usec;
filp_close(fp_write, NULL);
write_time = (write_end_time - write_start_time) * 1000000 + (write_end_time_u -
-- INSERT --
```

③编写 **Makefile** 文件，按“**i**”键进入 **Insert** 模式，在文件中写入如下代码，按“**Esc**”退出 **Insert** 模式，输入“：**wq**”保存：

```
ifneq ($(KERNELRELEASE),)
    obj-m := write_to_disk.o
else
    # 设置内核源码路径
    KERNELDIR ?= /usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64
    PWD := $(shell pwd)

    # 默认目标：调用内核的 make 命令来构建模块
    default:
        $(MAKE) -C $(KERNELDIR) M=$(PWD) modules
endif

.PHONY: clean

# 清理构建生成的文件
clean:
    -rm *.mod.c *.o *.order *.symvers *.ko
```

④编译并加载 **U** 盘内核模块：

```
[root@localhost usb_demo2]# make
make -C /usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64 M=/root/usb_demo2 modules
make[1]: 进入目录“/usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64”
  CC [M]  /root/usb_demo2/write_to_disk.o
  MODPOST /root/usb_demo2/Module.symvers
  CC [M]  /root/usb_demo2/write_to_disk.mod.o
  LD [M]  /root/usb_demo2/write_to_disk.ko
make[1]: 离开目录“/usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64”
[root@localhost usb_demo2]#
```

⑤测试写速率：

```
[root@localhost usb_demo2]# insmod write_to_disk.ko
[root@localhost usb_demo2]# dmesg | tail -n 3
[14686.182319] Start write_to_disk module...
[14690.531619] Writing to file costs 4347119 us
[14690.532038] Writing speed is 123 M/s
[root@localhost usb_demo2]#
```

⑥卸载模块：

```
[root@localhost usb_demo2]# rmmod write_to_disk.ko
[root@localhost usb_demo2]# dmesg | tail -n 3
[14690.531619] Writing to file costs 4347119 us
[14690.532038] Writing speed is 123 M/s
[14713.884804] Exit write_to_disk module...
[root@localhost usb_demo2]#
```

⑦ 编写内核模块测试硬盘的读速率。 和上述步骤一样，创建目录 **usb_demo3,cd** 到该目录下,创建 **read_from_disk.c** 文件和 **Makefile** 文件，在文件中分别写入如下内容，编译加载模块并测试即可：

```
[root@localhost usb_demo2]# cd ..
[root@localhost ~]# mkdir usb_demo3
[root@localhost ~]# cd usb_demo3
[root@localhost usb_demo3]# touch read_from_disk.c
[root@localhost usb_demo3]# vi read_from_disk.c
```

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/rtc.h>
#define buf_size 1024
#define read_times 524288
MODULE_LICENSE("GPL");
struct timeval tv;
static int __init read_disk_init(void)
{
struct file *fp_read;
char buf[buf_size];
int i;
int read_start_time;
int read_start_time_u;
int read_end_time;
int read_end_time_u;
int read_time;
loff_t pos;
printk("Start read_from_disk module...\n");
fp_read = filp_open("/home/tmp_file", O_RDONLY, 0);
if (IS_ERR(fp_read))
{
printk("Failed to open file...\n");
return -1;
}
do_gettimeofday(&tv);
read_start_time = (int)tv.tv_sec;
read_start_time_u = (int)tv.tv_usec;
pos = 0;
for (i = 0; i < read_times; i++)
{
kernel_read(fp_read, buf, buf_size, &pos);
}
do_gettimeofday(&tv);
read_end_time = (int)tv.tv_sec;
read_end_time_u = (int)tv.tv_usec;
filp_close(fp_read, NULL);
read_time = (read_end_time - read_start_time) * 1000000 + (read_end_time_u -
read_start_time_u);
printk(KERN_ALERT "Read file costs %d us\n", read_time);
printk("Reading speed is %d M/s\n", buf_size * read_times / read_time);
:
```

Make 编译成功

```
[root@localhost usb_demo3]# make
make -C /usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64 M=/root/usb_demo3 modules
make[1]: 进入目录“/usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64”
  CC [M]  /root/usb_demo3/read_from_disk.o
  MODPOST /root/usb_demo3/Module.symvers
  CC [M]  /root/usb_demo3/read_from_disk.mod.o
  LD [M]  /root/usb_demo3/read_from_disk.ko
make[1]: 离开目录“/usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64”
[root@localhost usb_demo3]#
```

结果如下

```
[root@localhost usb_demo3]# insmod read_from_disk.ko
[root@localhost usb_demo3]# dmesg -C
[root@localhost usb_demo3]# rmmod read_from_disk.ko
[root@localhost usb_demo3]# insmod read_from_disk.ko
[root@localhost usb_demo3]# dmesg |tail -n 20
[15045.818452] Exit read_from_disk module...
[15049.411478] Start read_from_disk module...
[15049.523843] Read file costs 112007 us
[15049.524212] Reading speed is 4793 M/s
[root@localhost usb_demo3]#
```

⑧IOZONE 工具测试，先查看确认当前环境：

```
[root@localhost usb_demo3]# free -m
               total        used        free      shared  buff/cache   available
Mem:           12122         661       10472          47        1318       11460
Swap:           2047           0        2047
[root@localhost usb_demo3]#
```

⑨执行测试：

```
iozone test complete.
Excel output is below:

"Writer report"
        "1"
"524288"    992287
"1048576"    993703
"2097152"    873230
"4194304"    871254
"8388608"    732635

"Re-writer report"
        "1"
"524288"    1342689
"1048576"    1170197
"2097152"    1157664
"4194304"    29701
"8388608"    28979

"Reader report"
        "1"
"524288"    2400452
"1048576"    2457782
"2097152"    2512494
"4194304"    1402694
"8388608"    1326639

"Re-Reader report"
        "1"
"524288"    2293291
"1048576"    2341093
"2097152"    2366310
"4194304"    1419464
"8388608"    1323553
```

SPOOLING 假脱机模拟

①打开 **FinalShell** 窗口，创建一个 **C** 文件，命名为 **spooling**，并打开 **spooling.c** 文件：

```
[root@localhost usb_demo3]# cd ..
[root@localhost ~]# mkdir exp8
[root@localhost ~]# cd exp8
[root@localhost exp8]# touch spooling.c
[root@localhost exp8]# vi spooling.c
```

② 按“i”键进入 Insert 模式，在 spooling.c 文件中写入如下代码，按“Esc”退出 Insert 模式，输入“：wq”保存：

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
struct pcb {
int id;
int status;
int firstaddr;
int length;
int outbufword;
}*PCB[3];
FILE *f;
struct req { //输出请求块
int reqname;//请求输出的用户进程名
int length;//要输出信息的长度
int addr;//输出信息在输出井中的位置
}reqblock[10];
int buffer[2][100];//为两个用户进程设置两个输出井，每个可存放 100 个信息
int c3=10;
int l1=1, l2=1;
int head=0, tail=0;
int t1, t2;
void request(int i) {
int j, length=0, m;
struct req *run;
if( (tail-head) == 10 ) {
PCB[i-1]->status =1;
return;
}
if(i==1) t1--;
else t2--;
run = &reqblock[tail%10];
run->reqname = i;
run->length = 0;
if(tail == 0) run->addr = 0;
else { int index = (tail-1)%10;
run->addr = reqblock[index].addr + reqblock[index].length;
}
for(m=0; m<100; m++)
{ if(buffer[i-1][m] == 0)
{ run->addr = m;
break;
"spooling.c" 123L, 2621B
```

③ 编译 spooling.c 文件，生成可执行文件 spooling，并执行：

```
[root@localhost exp8]# gcc spooling.c -o spooling
[root@localhost exp8]# ./spooling
How many work do process 1 want to do? 2
How many work do process 2 want to do? 3
process 2: 5 7 7 9 5 4 7 9 6 5 4 2 2 5 7 9 7 9 4 6 7 5 4
process 2: 2
process 2: 5 8 4
process 1: 3 1 2 7
process 1: 5 8 8 9 2 5 6 1 4 7 8
PID 2: 5 7 7 9 5 4 7 9 6 5 4 2 2 5 7 9 7 9 4 6 7 5 4
PID 2: 2
PID 2: 5 8 4
PID 1: 3 1 2 7
PID 1: 5 8 8 9 2 5 6 1 4 7 8
[root@localhost exp8]#
```

五、实验数据（现象）处理分析

Q1:插入内核模块提示没有对应内核，并且 usr/src/kernels 下是空的

```
[root@localhost usb_demo1]# insmod usb_detect.ko
insmod: ERROR: could not insert module usb_detect.ko: Invalid parameters
[root@localhost usb_demo1]# ls
Makefile  modules.order  Module.symvers  usb_detect.c  usb_detect.ko  usb_detect.mod  usb_detect.mod.c  usb_detect.mod.o  usb_detect.o
[root@localhost usb_demo1]# dmesg | tail -n 20
[   12.003868] [drm] Using command buffers with DMA pool.
[   12.003871] [drm] Atomic: yes.
[   12.285030] fbcon: svgadrmfb (fb0) is primary device
[   12.286404] Console: switching to colour frame buffer device 100x37
[   12.298213] [drm] Initialized vmwgfx 2.18.0 20200114 for 0000:00:0f.0 on minor 0
[   12.704899] NET: Registered protocol family 40
[   16.096453] bridge: filtering via arp/ip/ip6tables is no longer available by default. Update your scripts to load br_netfilter if you need this.
[   16.109812] Bridge firewalling registered
[  208.339226] e1000: ens33 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: None
[  208.341422] IPv6: ADDRCONF(NETDEV_CHANGE): ens33: link becomes ready
[ 2068.677147] usb_detect: loading out-of-tree module taints kernel.
[ 2068.677953] usb_detect: module verification failed: signature and/or required key missing - tainting kernel
[ 2068.679027] usb_detect: disagrees about version of symbol usb_alloc_urb
[ 2068.679458] usb_detect: Unknown symbol usb_alloc_urb (err -22)
[ 2068.679921] usb_detect: disagrees about version of symbol usb_free_urb
[ 2068.680350] usb_detect: Unknown symbol usb_free_urb (err -22)
[ 2068.680896] usb_detect: disagrees about version of symbol usb_put_dev
[ 2068.681331] usb_detect: Unknown symbol usb_put_dev (err -22)
[ 2068.681954] usb_detect: disagrees about version of symbol usb_get_dev
[ 2068.682409] usb_detect: Unknown symbol usb_get_dev (err -22)
[root@localhost usb_demo1]# [root@localhost usb_demo1]# ls
bash: [root@localhost: 未找到命令
```

A1：需要通过安装 dnf 工具，产生内核源码，如下

/usr/src/kernels/5.10.0-237.0.0.136.oe2203sp4.x86_64

```
[root@localhost usb_demo1]# sudo dnf install kernel-devel
Last metadata expiration check: 0:19:11 ago on 2024年11月27日 星期三 10时16分38秒.
Dependencies resolved.
================================================================================
 Package                    Architecture                    Version
================================================================================
Installing:
 kernel-devel               x86_64                          5.10.0-237.0

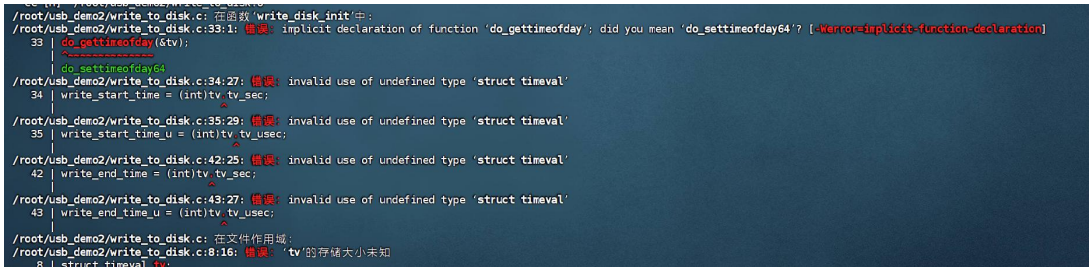
Transaction Summary
================================================================================
Install  1 Package
```

Q2：插入 U 盘无显示



A2：需要将设备连接到虚拟机而非宿主机

Q3：产生如下报错



A3：查阅资料发现 **do_gettimeofday** 可能已经被废弃。用 **struct timespec64** 取代旧的时间结构

六、实验结论

通过本次实验，我深入了解了 **openEuler** 操作系统中的设备管理与 **SPOOLING** 原理。实验过程中，我们编写并调试了 **USB** 驱动模块，成功实现了对 **U** 盘的插拔检测，并验证了内核模块的加载与卸载过程。在内核文件编写部分，测试了硬盘的读写速率，并通过 **I/O** 测试工具 **IOZONE** 对性能进行了评估。最后，在模拟 **SPOOLING** 假脱机功能时，我们成功实现了任务队列的处理，模拟了打印任务的管理和调度。

实验中遇到的一些问题，如内核模块无法加载和 **U** 盘插入不显示，通过查阅文档和调整配置文件解决了。整体而言，本次实验帮助我加深了对操作系统设备管理和任务调度机制的理解，并提高了我的内核模块编写与调试能力。

七、教师批阅意见：   成绩评定：   教师签字： 年　　月　　日
八、备注：